

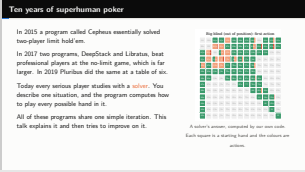
Timing plan for 60 minutes. Opening and rules 5 · Games, equilibria and the linear program 9 · Regret minimisation and CFR 9 · Live demonstration 12 · The new method 13 · Results, limits and wrap-up 8. That leaves about four minutes for questions during the talk.

If running late, shorten the demonstration to its five minute version (see `app/DEMO_NOTES.md`) and give the slide "Making it robust and fast" one sentence.

Opening line: "Ten years ago the best poker players in the world lost to programs whose core is a few lines of arithmetic repeated a few billion times. This talk is about that arithmetic, read as numerical analysis, and about what a numerical analyst can add to it."

The talk alternates between poker and mathematics, and our own contribution comes in the last third. On the plan slide the poker parts are green, the mathematics blue and the new method orange. Every number that we computed comes from the Python solver in the repository. The more detailed version of this deck, with all citations and extra slides, is `talk-detailed.pdf`.

Ten years of superhuman poker



The common core of all these programs is an iterative method, counterfactual regret minimisation, applied to a very large saddle-point problem. Libratus and DeepStack add abstraction (shrinking the game) and subgame solving (solving again the part of the game you are actually in), but the inner loop is still a method of the CFR family.

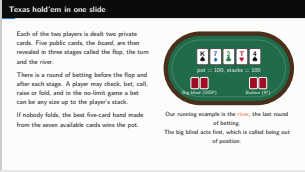
Commercial solvers such as PioSolver, GTO+ and GTO Wizard are what human players use. You give the board, the two ranges and the allowed bet sizes, and the program computes an equilibrium of that subgame. The picture on the right is exactly such an output, produced by our solver for the river situation that we use for the whole talk.

References if asked: Bowling et al., Science 2015. Moravčík et al., Science 2017. Brown and Sandholm, Science 2018 and 2019.

The plan: poker ↔ math



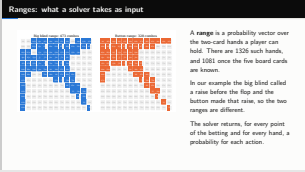
Say what the audience will get. First, enough poker to read the output of a solver. Second, the mathematics of why CFR converges and why it is fast in practice. Third, a live demonstration in which they can ask for any change to the hand. Fourth, a new method from the numerical optimisation toolbox, with honest results: where it wins by orders of magnitude, and where it does not yet.



Keep this light, since most of the audience has played or watched poker. The key structural fact is that the board is public and the two hole cards are private. A strategy must therefore be a function of the public history and your own cards. You do not know the opponent's cards, only a probability distribution over them, which poker players call a **range**.

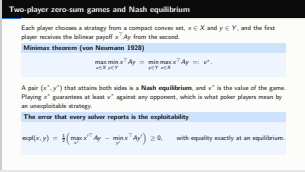
The river situation is fixed for the whole talk: board K♠ 7♢ 2♣ T♡ 4♠, pot 100, both players have 100 behind. The big blind may check or bet 33% or 75% of the pot or go all-in. Facing a bet you may fold, call or go all-in. The live demonstration uses exactly this table.

Ranges: what a solver takes as input



The 13 by 13 grid is the standard poker picture. Pairs are on the diagonal, suited hands above it and offsuit hands below it. Each cell is a class of hands, for example AKs stands for 4 combinations. Mathematically the input is two nonnegative vectors $r_1 \in \mathbb{R}^{n_1}$ and $r_2 \in \mathbb{R}^{n_2}$. Two hands that share a card cannot both be dealt, which is called card removal, and it is a 0/1 compatibility matrix C between the ranges. Both will appear in the formulas later.

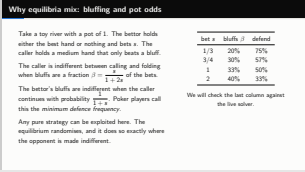
The width of each bar shows how many combinations are still possible given the board. The king of spades is on the board, so there are fewer hands with a king.



For an applied mathematics audience this is a convex-concave saddle-point problem with a bilinear coupling, and the exploitability is the primal-dual gap. Everything later is a method for driving this gap to zero. Its unit is chips per hand.

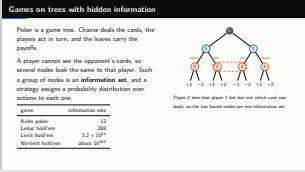
Practical solvers stop at an exploitability of a fraction of a percent of the pot. We will push to 10^{-13}. Our numbers are in chips with a pot of 100, so 10^{-2} chips is 10^{-4} of the pot.

Why equilibria mix: bluffing and pot odds



Derivation, to say and not to write. For the caller, calling wins $1 + s$ against a bluff and loses s against a value bet. The caller is indifferent when $\beta(1 + s) = (1 - \beta)s$, which gives $\beta = s/(1 + 2s)$. For the bettor with nothing, betting wins the pot 1 when the caller folds and loses s when called, and checking wins nothing. The bettor is indifferent when $(1 - c) \cdot 1 = c\,s$, which gives $c = 1/(1 + s)$.

The mathematical point: at an equilibrium every action played with positive probability is a best response. This is complementary slackness in the linear program that we are about to write down. Mixing happens exactly where reduced costs vanish.



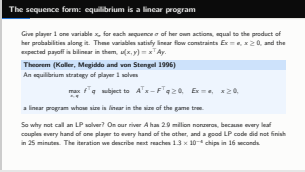
Leduc hold'em is the standard research benchmark. It has a deck of six cards, one private card each, one public card, and two betting rounds. It has 288 information sets and we use it as our small test problem.

The size column is the reason the subject exists. Nobody can write down no-limit hold'em. Real systems solve an abstraction in which similar hands and bet sizes are merged, and then solve subgames like our river again during play.

We assume perfect recall: a player never forgets what he knew or did. Sizes are from Bowling et al. 2015 and Johanson 2013.

Kuhn and Leduc were counted by our code.

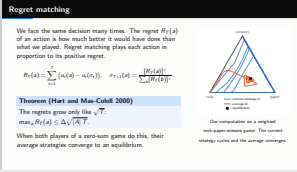
The sequence form: equilibrium is a linear program



This is the key modelling step. Mixed strategies over the pure strategies of the normal form would have exponentially many variables. Realisation plans have one variable per sequence, and perfect recall makes the payoff bilinear in them.

E and F are very sparse, with one row per information set and entries ± 1, and they encode flow conservation on a tree. A is where the poker lives. The program has 17,626 unknowns on our river. Say that the dual variables q are the values of the opponent's information sets, and remember them: they are values of information sets, the same quantities that CFR computes in every iteration. This is how we will warm-start the duals later.

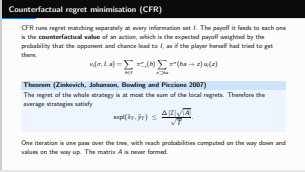
The LP run used HiGHS through SciPy with default settings on one core and we stopped it after 25 minutes. On the smaller test problems the LP is perfectly fine: 0.04 s on Leduc and 1.6 s on a small river. The DCFR figure is for 5,000 iterations.



Idea of the proof. The regret vector satisfies $\|[R_T]^+\|^2 \le \|[R_{T-1}]^+\|^2 + \Delta^2|A|$ because the cross term $\langle [R_{T-1}]^+, r_T\rangle$ vanishes by the choice of σ_T: the instantaneous regret is orthogonal to the current positive regrets. Sum and take square roots. This is Blackwell approachability, and Δ is the range of the payoffs.

The folk theorem: if both players have average regret at most ε then the average strategies are a 2ε-equilibrium, because $\max_{x'} x'^{\top} A\bar{y} - \min_{y'} \bar{x}^{\top} A y' \le 2\varepsilon$.

In the figure the limit is $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$.

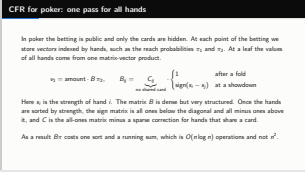


The decomposition is the whole trick. A global regret over an exponentially large strategy space is bounded by a sum of small local regrets, and each of them is minimised by plain regret matching. This is why CFR scales: the cost of an iteration is linear in the tree and so is the storage, two numbers for each information set and action.

In the language of linear algebra, one CFR iteration computes Ay and $A^{\top}x$ for the current realisation plans by walking the tree.

These are the same two products that any first-order saddle-point method needs.

└CFR for poker: one pass for all hands



This is the bridge between the tree picture and the implementation. The information sets at a public node are indexed by the private hand of the player to act, so all information sets at this node form a vector and every CFR operation becomes a vector operation.

This is also the poker-specific numerical linear algebra. Each terminal block is low rank plus sparse. The fold block is rank one minus a sparse correction. The showdown block, after sorting, is a sign matrix, which is semiseparable (think of the inverse of a bidiagonal matrix), times the compatibility mask. The trick is due to Johanson et al. 2011.

The public tree itself is drawn live in the demonstration, so it is not on a slide any more. It has 12 decision points and 21 leaves, and one iteration takes about a millisecond in the browser. Keep in mind that this structure is what a scalable Newton method would have to exploit, which comes back at the end of the talk.

Four variants of one loop

Modern solvers change how CFR weights the old regrets and how it averages the strategies.

method	what changes	a numerical analyst would say
CFR$^+$	Negative regrets are reset to zero and later strategies get more weight in the average.	projection onto the nonnegative orthant
DCFR	Old regrets and old strategies are multiplied by factors below one.	forgetting factors that damp the transient
PCFR$^+$	The strategy is computed from the regrets plus a prediction of the next regret.	optimism, as in the extragradient method

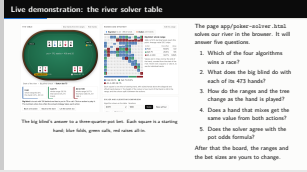
The proven rate is still $T^{-1/2}$ for all four. In practice the modern variants are much faster, and we will now race them against each other on a real hand.

CFR$^+$ (Tammelin 2014) is what solved limit hold'em. DCFR (Brown and Sandholm 2019) with parameters $(1.5, 0, 2)$ is the default in many solvers: positive regrets are multiplied by $t^{1.5}/(t^{1.5}+1)$, negative ones by $1/2$, and the average by $(t/(t+1))^2$. PCFR$^+$ (Farina, Kroer and Sandholm 2021) uses the last instantaneous regret as its prediction and connects regret matching to optimistic mirror descent.

All four also update the two players in turn, so that the second player already sees the new strategy of the first, which is Gauss-Seidel in place of Jacobi.

The observed rates are between T^{-1} and $T^{-3/2}$ for the averages, while the last iterate oscillates. Explaining this gap to the theory is an open question. The rates are still sublinear, and that is the gap our method addresses.

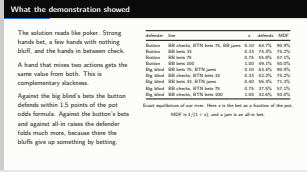
Live demonstration: the river solver table



Switch to the browser now. The full script with what to click and what to say is in `app/DEMO_NOTES.md` (also as a PDF), and the same steps are in the Demo guide inside the page. Budget twelve minutes.

Short version of the script. (1) Load the page, which solves the spot with DCFR in two seconds. Point at the table, the grid, the tree and the chart. (2) Press Race all four. After 1000 iterations CFR is at about 0.11% of the pot, PCFR$^+$ at 0.009%, CFR$^+$ at 0.007% and DCFR at 0.002%. (3) Read the big blind's grid: check 82%, bet a third of the pot 16%, bet three quarters 2%. Value bets and bluffs. (4) Give the big blind three sevens: it checks every time, and after the button bets 75 it raises all-in every time. (5) After check and bet 75, select ace-nine offsuit: it folds 63% and calls 37%, and both actions are worth 0 chips to within the accuracy of the solve. (6) After the big blind bets 33, the button folds 26%, and the formula says at most 25%. After a check and an all-in the big blind folds 68%, far more than the formula's 50%. (7) Change the river card to the nine of hearts and solve again. Then take requests from the audience.

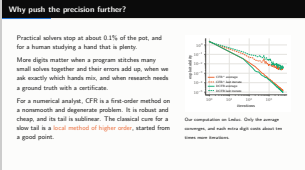
If the browser fails, the static grids are in `talk-detailed.pdf`, slides 18 and 19.



This is a consistency check of the whole pipeline. When the big blind bets, the button defends 74.3% against 75.2%, 55.8% against 57.1% and 49.1% against 50.0%, so the toy formula is almost exact. When the button bets after a check, the big blind folds much more. We checked that the button's weakest betting hands mix between checking and going all-in, so the indifference that fixes the defence frequency is "value of the all-in equals value of checking, which is positive" and not "value of the all-in equals zero". The all-in raises (the rows with $s = 0.10$ and $s = 0.40$) fail the formula for the same reason. A bluff-raise gives up the value of simply calling, and the raising range is mostly strong hands.

The numbers in the table come from the exact equilibrium computed by the method of the next part, so they can differ in the last digit from what the browser showed after 1000 iterations.

Why push the precision further?

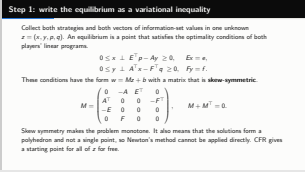


Make the case without overselling. Precision matters for machines (solving subgames again, evaluation) and for science (knowing the exact equilibrium, for example the exact bluffing frequencies, or whether a hand mixes at all).

The analogy for this audience: CFR is like a stationary iterative method. It is fantastic for the first digits and hopeless for the last ones, and the obvious idea is to switch to a Newton-type method near the solution.

Numbers for Leduc: CFR$^+$ reaches 9.2×10^{-7} after 50,000 iterations and 130 seconds. The curves are on a backup slide.

Step 1: write the equilibrium as a variational inequality



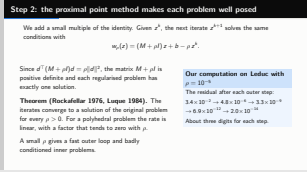
This is a mixed linear complementarity problem. The reduced cost $E^\top p - Ay$ of a sequence is its counterfactual regret scaled by reach, and complementarity says "play only actions with zero regret". So CFR and this system talk about the same quantities: the CFR average strategies are x and y, and its best-response values are p and q.

Skewness: the x-y block is the zero-sum coupling. The (x, p) and (y, q) blocks are pairs of constraints and multipliers, arranged with signs so that $M^\top = -M$.

Non-uniqueness is already visible in Kuhn poker, where the equilibria form a one-parameter family (backup slide).

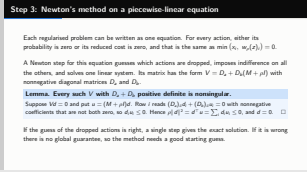
If asked what we tried first: Anderson acceleration of the CFR$^+$ map, vector extrapolation of the averages, and Levenberg-Marquardt on the optimality system. None of them beat DCFR on Leduc. The CFR map is piecewise linear with non-isolated fixed points, and the acceleration theory needs a smooth contraction. The details are in `talk-detailed.pdf`.

Step 2: the proximal point method makes each problem well posed



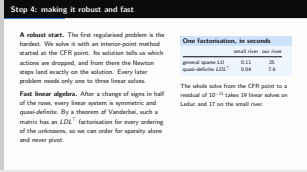
The proximal term fixes the non-uniqueness. Each subproblem has exactly one solution, so Newton's method has a well defined and nonsingular target. For polyhedral monotone problems the inverse operator is upper Lipschitz (Robinson 1981), which gives linear convergence of the proximal point method with a factor that tends to zero as $\rho \to 0$.

The trade-off is the whole story of the last part of the talk. A small ρ makes the outer loop fast and the inner problems nearly as ill posed as the original one, since $\|(M + \rho I)^{-1}\| \leq 1/\rho$. There is a backup slide with the rate.



The min-function turns complementarity into a piecewise-linear equation. Semismooth Newton on it is exactly the primal-dual active-set method (Hintermüller, Ito and Kunisch 2002). The lemma is the reason for the proximal term: without ρ we have $d^\top M d = 0$ and the argument fails. Newton matrices of the unregularised problem are singular at degenerate solutions, which is what we saw with Levenberg-Marquardt.

The lemma also covers the Fischer-Burmeister reformulation, which has a smooth merit function and a globally convergent damped Newton method. We tried it first. It worked on Leduc and on a small river with 50 to 70 linear solves, but on the river of the demonstration it took steps of size 10^{-3} for dozens of iterations. That pushed us to the interior-point start on the next slide.

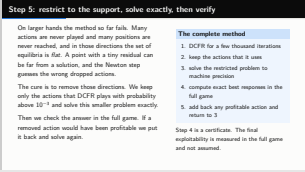


The first problem is the hardest because many actions enter or leave the support. The division of labour mirrors what commercial LP solvers do, interior point and then crossover to a vertex, but inside a proximal loop and with a game-theoretic warm start. We use Mehrotra's predictor-corrector method. In our runs the interior-point phase took 6 to 8 factorisations on Leduc and the small river, the crossover a few active-set steps, and every later proximal step 1 to 3 steps. We use $\rho = 10^{-5}$ throughout.

Quasi-definite means the block form $\left(\begin{smallmatrix} P & B \\ B^\top & -N \end{smallmatrix}\right)$ with P and N positive definite, after grouping (x, q) and (y, p). Vanderbei's theorem (1995) lets us choose a fill-reducing ordering before seeing the numbers, which is the same reason interior-point codes like quasi-definite systems. The timings compare SuperLU with QDLDL on one core.

A caveat: with only $\pm 10^{-5}$ on the diagonal, as in the pure active-set systems, static pivoting is too unstable. There we use a pivoted sparse LU, which is cheap because the active-set system lives on the support only.

Step 5: restrict to the support, solve exactly, then verify

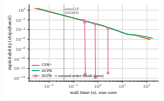


The verification loop is column generation, the same idea as the double-oracle algorithms for games (McMahan, Gordon and Blum 2003, Bošanský et al. 2014). We also keep only the positions that both players can still reach with the kept actions. What is new is the combination with a support predicted by CFR and an exact second-order solve of the restricted problem.

Poker helps here. Equilibria are mostly pure, because mixing happens only at indifference. In the exact Leduc equilibrium about half of the reachable pairs of information set and hand play a single action, and on the small river 71%. About half of all sequences are outside the support, and the dense payoff blocks shrink to the hands that actually reach each leaf.

On our problems Leduc needed 2 rounds, the small river 1, and the river of the demonstration 3, with 16 actions added back in total. The restricted problem had about 35% of the unknowns of the full one.

Result on the benchmark game, Leduc hold'em

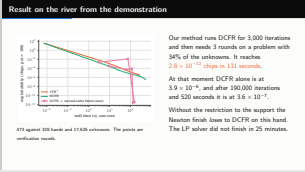


Each pink curve starts where the DCFR warm start stopped, after 100, 300 or 1000 iterations. The points are verification rounds. Starting from 100 DCFR iterations is enough and costs one extra round. Starting later only costs DCFR time. The run used 2 rounds and 26 linear solves.

The value of the game that we obtain, $-0.085606424078\ldots$ chips for the first player, agrees with the LP in all printed digits.

Be upfront about the LP. Leduc has 964 unknowns and a simplex method finishes in 40 ms. The question is what happens when the LP does not fit, which is the next slide.

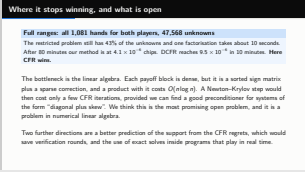
Result on the river from the demonstration



This is the slide that took the longest to earn. The first version of the method, Newton on the full sequence-form problem, worked on Leduc and the small river and lost to DCFR here, with 3.2×10^{-5} after 528 seconds. The crossover from the interior point kept landing on a wrong active set, because the equilibrium set is flat in directions that do not matter, and with $\rho = 10^{-5}$ an error in those directions is amplified by $1/\rho$. A larger ρ made the active-set steps work and the outer loop crawl. Anderson acceleration of the proximal map helped, to 8×10^{-7} in 8 minutes, and then stalled.

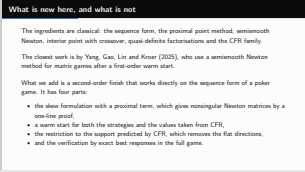
Restricting to the support predicted by CFR and verifying afterwards removes exactly those directions. 16 actions were added back in total. Starting from 1,000 DCFR iterations gives 3.8×10^{-12} in 152 seconds.

On the small river (147 against 139 hands) the method reaches 1.1×10^{-12} in 0.95 seconds, DCFR is at 5.4×10^{-6} after 9 seconds, and the LP takes 1.6 seconds. This is on a backup slide.



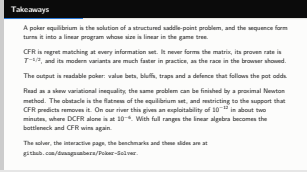
The run with full ranges is the honest boundary of the method as implemented. The restriction still removes more than half of the unknowns, but each factorisation of the remaining system with 20,000 unknowns takes about 10 seconds because the payoff blocks between the hands that remain in play are dense. The first round needed 201 solves and 50 minutes, the second 116, and the second did not reach machine precision (the proximal loop stalled at a residual of 5×10^{-7}). DCFR does 40,000 iterations in 10 minutes.

The research directions are natural ones for this club. The first is the one I would bet on: the structure that makes CFR fast (sorting, running sums, card-removal corrections) is exactly the structure that a fast direct or preconditioned iterative solver for the Newton systems would need. Hierarchical factorisations of the blocks are a candidate.



Be precise about novelty. The general idea of a first-order warm start followed by semismooth Newton for zero-sum games was proposed for matrix games by Yang, Gao, Lin and Kroer in December 2025 (arXiv 2512.12910). Their Newton system lives on the normal form, they use Douglas-Rachford splitting and a PRM$^+$ warm start, and they do not treat the structure of extensive-form games.

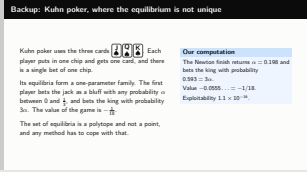
To our knowledge no solver based on CFR hands over to a second-order method for the last digits. Early work solved small abstracted games exactly as linear programs (Gilpin and Sandholm 2005). The commercial and research poker solvers that we know of (PioSolver, GTO+, Libratus, DeepStack) are first-order.



End with the invitation. The open problem on the earlier slide is a natural one for this club: a preconditioner for shifted skew systems whose off-diagonal blocks are a sorted sign matrix plus a sparse correction.

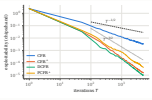
If there is time left, go back to the browser and take requests: another board, another range, other bet sizes.

Backup: Kuhn poker, where the equilibrium is not unique



Kuhn poker is the poker analogue of a two by two test matrix. Jacobians at a solution are singular, and the proximal term picks one member of the family.

Backup: the first-order methods on Leduc hold'em



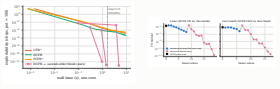
The curves show the exploitability of the average strategy for our implementation.

After 5,000 iterations and 14 seconds DCFR reaches 1.0×10^{-3}. After 50,000 iterations and 130 seconds CFR$^+$ reaches 9.2×10^{-7} and DCFR 1.6×10^{-6}.

Each extra digit costs about ten times more iterations.

Exploitability is in chips per hand, and the antes in Leduc are 1 chip.

Backup: a small river, and every linear solve of one exact solve

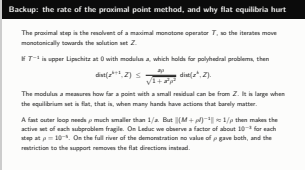


On the left is a river with 147 against 139 hands. Our method reaches 1.1×10^{-12} chips in 0.95 seconds, DCFR is at 5.4×10^{-4} after 9 seconds, and the LP takes 1.6 seconds. On the right is the residual after every linear solve. The blue points are interior-point iterations and the pink points are Newton steps.

The crossover lands on the exact solution of the first regularised subproblem, whose residual is $\rho\|z^1 - z^0\|$, hence the jump. After that each proximal step is solved exactly in one to three steps.

Backup: the rate of the proximal point method, and why flat equilibria hurt

Backup for questions about the outer loop. Rockafellar 1976, Luque 1984, Robinson 1981.



Backup: the rate of the proximal point method, and why flat equilibria hurt

The proximal step is the resolvent of a maximal monotone operator T, so the iterates move monotonically towards the solution set Z.

If T^{-1} is upper Lipschitz at 0 with modulus a, which holds for polyhedral problems, then

$$\operatorname{dist}(x^{k+1}, Z) \le \frac{a}{\sqrt{1 + a^2\rho^2}} \operatorname{dist}(x^k, Z).$$

The modulus a measures how far a point with a small residual can be from Z. It is large when the equilibrium set is flat, that is, when many hands have actions that barely matter.

A fast outer loop needs ρ much smaller than $1/a$. But $\|(M + \rho I)^{-1}\| \approx 1/\rho$ then makes the active set of each subproblem fragile. On Leduc we observe a factor of about 10^{-4} for each step at $\rho = 10^{-4}$. On the full river of the demonstration no value of ρ gave both, and the restriction to the support removes the flat directions instead.